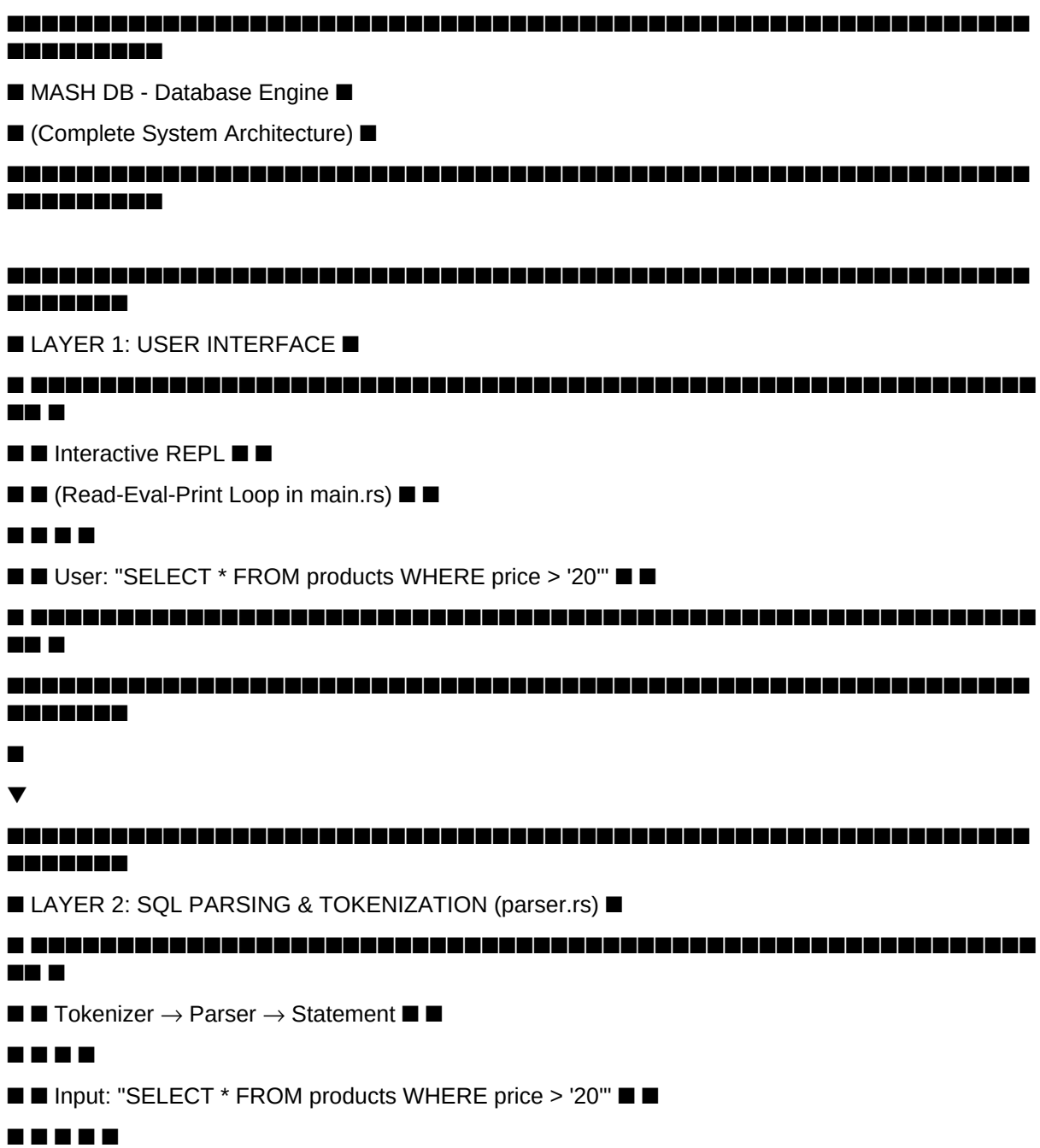
Mash DB - Complete Architecture & Structure Diagrams

System Architecture Overview

```
```



■ ■ ■ ■ ■

■ ■ ■■■ parse: GROUP BY columns, WHERE, etc ■ ■

■ ■ ■ ■ ■

■ ■ ■■■ Output: Statement::Select { ... } ■ ■

■ ■■
■■ ■

■■■
■■■■■■■

■

▼

■■■
■■■■■■■

■ LAYER 3: STATEMENT EXECUTION (main.rs) ■

■ ■■
■■ ■

■ ■ execute_statement(Statement) → Result ■ ■

■ ■ ■ ■

■ ■ Match statement type: ■ ■

■ ■ ■■■ INSERT → Call table.insert(values) ■ ■

■ ■ ■■■ SELECT → Call table.select_where_complex() ■ ■

■ ■ ■■■ UPDATE → Call table.update() ■ ■

■ ■ ■■■ DELETE → Call table.delete() ■ ■

■ ■ ■■■ CREATE → Create new table in schemas ■ ■

■ ■ ■■■ DROP → Remove table from schemas ■ ■

■ ■ ■■■ ALTER → Modify table structure ■ ■

■ ■■
■■ ■

■■■
■■■■■■■

■

▼

■■■
■■■■■■■

■ LAYER 4: TABLE OPERATIONS (table.rs) ■

■ ■■
■■ ■

■ ■ Core Data Manipulation ■ ■

■ ■ ■ ■

■ ■ ■■ ■ ■

■ ■ ■ Table Structure: ■ ■ ■

■ ■ ■ ■■ schema: Vec [Columns] ■ ■ ■

■ ■ ■ ■■ rows: Vec [Data] ■ ■ ■

■ ■ ■ ■■ id_index: BTreeMap [Fast lookups] ■ ■ ■

■ ■ ■ ■■ username_index: BTreeMap ■ ■ ■

■ ■ ■ ■■ email_index: BTreeMap ■ ■ ■

■ ■ ■■ ■ ■

■ ■ ■ ■

■ ■ ■■ ■ ■

■ ■ ■ Row Structure: ■ ■ ■

■ ■ ■ ■■ id: u32 [Primary key] ■ ■ ■

■ ■ ■ ■■ username: String [Fixed field] ■ ■ ■

■ ■ ■ ■■ email: String [Fixed field] ■ ■ ■

■ ■ ■ ■■ extras: HashMap ■ ■ ■

■ ■ ■ [Dynamic columns beyond id/username/email] ■ ■ ■

■ ■ ■■ ■ ■

■ ■ ■ ■

■ ■ Operations: ■ ■

■ ■ ■■■ insert(row) → adds row + updates indexes ■ ■

■ ■ ■■■ select_where_complex() → queries with filters ■ ■

■ ■ ■■■ update(id, updates) → modifies row + indexes ■ ■

■ ■ ■■■ delete(id) → removes row + cleans indexes ■ ■

■ ■ ■■■ get_value(row, column) → dynamic accessor ■ ■

■ ■ ■■■ compute_aggregate() → COUNT/SUM/AVG/etc ■ ■

■ ■■ ■

■■

■

▼

■■

■ LAYER 5: INDEXING & LOOKUP (B-Tree) ■

■ ■■ ■

■ ■ Fast Lookup Structures ■ ■

■ ■ ■ ■

■ ■ Primary Index (id): ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ BTreeMap ■ ■ ■

■ ■ ■ 1 → Row { alice, ... } ■ ■ ■

■ ■ ■ 2 → Row { bob, ... } ■ ■ ■

■ ■ ■ 3 → Row { charlie, ... } ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ ■

■ ■ Secondary Index (username): ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ BTreeMap> ■ ■

■ ■ ■ "alice" → [1] ■ ■ ■

■ ■ ■ "bob" → [2] ■ ■ ■

■ ■ ■ "charlie" → [3] ■ ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ ■

■ ■ Performance: ■ ■

■ ■ ■■ Index hit: O(log n) ✓ Fast ■ ■

■ ■ ■■ No index: O(n) Full scan ■ ■

■ ■■
■■ ■

■■■
■■■■■■■

■

▼

■■■
■■■■■■■

■ LAYER 6: STORAGE MANAGEMENT - PAGER (pager.rs) ■

■ ■■
■■ ■

■ ■ Page-based Storage with Memory Management ■ ■

■ ■ ■ ■

■ ■ Memory: [Page 1] [Page 2] [Page 3] ... [Page N] ■ ■

■ ■ ▲ ▲ ▲ ▲ ■ ■

■ ■ ■ ■ ■ ■■ On-disk cache ■ ■

■ ■ ■■■■■■■■■■■■■■■■■ ■ ■

■ ■ All in-memory (current design) ■ ■
■ ■ ■ ■
■ ■ Format: Binary serialization (bincode) ■ ■
■ ■ Size: Configurable per page ■ ■
■ ■■ ■
■■
■
▼
■■
■ LAYER 7: PERSISTENCE (File System) ■
■ ■■ ■
■ ■ Binary JSON Storage Files ■ ■
■ ■ ■ ■
■ ■ users.json ← Stores user data (id, user, email) ■ ■
■ ■ products.json ← Stores products (custom schema) ■ ■
■ ■ orders.json ← Stores orders (custom schema) ■ ■
■ ■ stores.json ← Stores stores (custom schema) ■ ■
■ ■ schemas.json ← Stores all table schemas ■ ■
■ ■ ... ← One file per table ■ ■
■ ■ ■ ■
■ ■ Write Flow: RAM → bincode serialization → disk file ■ ■
■ ■ Read Flow: disk file → bincode deserialization → RAM ■ ■
■ ■■ ■
■■
```

---

# Data Flow Diagrams

## Flow 1: SELECT Query Execution
```

```
User Input:
"SELECT * FROM products WHERE price > '20' ORDER BY name LIMIT 10"
■
▼
■■■■■■■■■■■■■■■■■■■■■■
■ 1. TOKENIZE ■ → ["SELECT", "*", "FROM", "products", ...]
■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 2. PARSE ■
■ Parse tokens into ■
■ Statement::Select ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 3. GET TABLE ■
■ Load "products" table ■
■ from schema registry ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 4. FILTER ROWS ■
■ Apply WHERE clause: ■
■ price > '20' ■
■ ■
■ Results: 2 rows ✓ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

■ 5. SORT ■

■ ORDER BY name ■

■ (alphabetical sort) ■

■ ■

■ Results: [A→Z] ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 6. LIMIT ■

■ LIMIT 10 ■

■ (take first 10) ■

■ ■

■ Results: 2 rows ✓ ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 7. RETURN RESULTS ■

■ ■

■ (1, Widget, 19.99, ...) ■

■ (2, Gadget, 29.99, ...) ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

```

# Flow 2: INSERT Query Execution

```

User Input:

"INSERT INTO products VALUES (3, 'Doohickey', '34.99', '75', 'Misc')"

■

▼

■■■■■■■■■■■■■■■■■■■■■

■ 1. TOKENIZE ■ → ["INSERT", "INTO", "products", ...]

■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 2. PARSE ■

■ Extract values from ■

■ VALUES clause: ■

■ [3, Doohickey, ...] ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 3. GET TABLE SCHEMA ■

■ From schema registry: ■

■ [id, name, price, ■

■ stock, category] ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 4. CREATE ROW ■

■ Row::from_values() ■

■ Maps values→columns ■

■ { ■

■ id: 3, ■

■ username: null, ■

■ email: null, ■

■ extras: { ■

■ "name": "Doohi..",■

■ "price": "34.99", ■

■ "stock": "75", ■

■ "category": "Misc"■

■ } ■

■ } ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 5. INSERT INTO TABLE ■

■ Add row to rows vec ■

■ Update id index ■

■ (3 → row position) ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 6. PERSIST TO DISK ■

■ Serialize to ■

■ products.json ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■

▼

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ 7. RETURN STATUS ■

■ ■

■ "Executed." ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

```

# Flow 3: Dynamic Schema Creation

```

User Input:

"CREATE TABLE inventory (id, sku, description, quantity, warehouse)"

■

▼

■■■■■■■■■■■■■■■■■■■■■

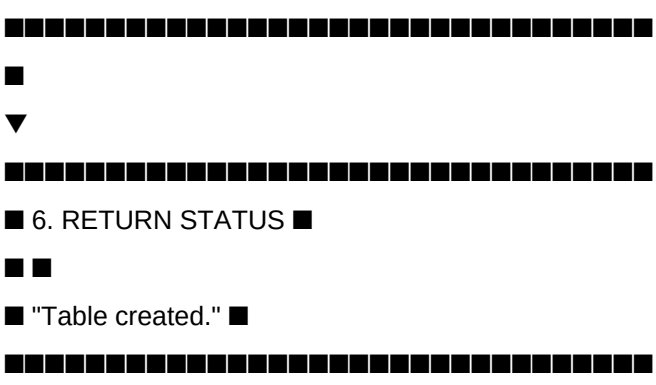
■ 1. PARSE ■ → Statement::CreateTable

■■■■■■■■■■■■■■■■■■■■■

■

▼

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 2. EXTRACT COLUMN NAMES ■
■ [id, sku, description, ■
■ quantity, warehouse] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 3. STORE IN SCHEMA REGISTRY ■
■ schemas.json: ■
■ { ■
■ "inventory": [ ■
■ "id", "sku", ■
■ "description", ■
■ "quantity", ■
■ "warehouse" ■
■ ] ■
■ } ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 4. CREATE EMPTY TABLE ■
■ Table { ■
■ schema: [...], ■
■ rows: [], ■
■ indexes: empty ■
■ } ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 5. PERSIST SCHEMA ■
■ Save to schemas.json ■
■ Create inventory.json ■
```




```

---

# Module Dependencies

```
MAIN.RS (CLI & Statement Execution)
  ├─ Uses: parser.rs (parse_insert, parse_select, parse_update, etc.)
  ├─ Uses: table.rs (Table, Row structures)
  ├─ Uses: pager.rs (Page management)
  ├─ Manages: schema_registry HashMap<String, Table>
  ├─ Maintains: default_table: Table
  ├─ Operations: INSERT, SELECT, UPDATE, DELETE, CREATE, DROP, ALTER
  └─ State: All open tables in memory

PARSER.RS (SQL Parsing)
  ├─ Input: Raw SQL strings from REPL
  ├─ Tokenizes: SQL into tokens
  ├─ Parses: Tokens into Statement enum
  ├─ Validates: SQL syntax
  └─ Output: Statement types
     ├─ Statement::Insert { table_name, values: Vec<Row> }
     ├─ Statement::Select { ... where, order_by, limit ... }
     ├─ Statement::Update { ... }
     ├─ Statement::Delete { ... }
     ├─ Statement::CreateTable { name, columns }
     └─ Statement::DropTable { name }
```
```

■ ■■ Statement::AlterTable { ... }

■■ State: Stateless (pure functions)

TABLE.RS (Data Structure & Operations)

■■ Row struct:

■ ■■ id: u32 (fixed)

■ ■■ username: String (fixed)

■ ■■ email: String (fixed)

■ ■■ extras: HashMap (dynamic)

■■ Table struct:

■ ■■ schema: Vec (column definitions)

■ ■■ rows: Vec (data storage)

■ ■■ id_index: BTreeMap (fast id lookups)

■ ■■ username_index: BTreeMap (fast username lookups)

■ ■■ email_index: BTreeMap (fast email lookups)

■■ Operations:

■ ■■ insert(&mut; self, row: Row)

■ ■■ select_where_complex(&self;, conditions) → Vec

■ ■■ update(&mut; self, id: u32, updates: HashMap)

■ ■■ delete(&mut; self, id: u32)

■ ■■ get_value(&row;, column) → Option

■ ■■ compute_aggregate(...) → Result

■■ Uses: BTreeMap for indexing

PAGER.RS (Storage Management)

■■ Page structure:

■ ■■ data: Vec (rows in page)

■ ■■ id: u64 (unique page id)

■■ Pager structure:

■ ■■ pages: HashMap

■ ■■ current_page_id: u64

■ ■■ table_name: String

■■ Operations:

■ ■■ add_row_to_page(row)

■ ■■ get_page(id) → Page

■ ■■ flush_to_disk()

```
■■ I/O: Serializes to .json files

COLUMN.RS (Column Definitions - Optional)
■■ Not currently used in main flow
```

Runtime State Diagram

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ RUNTIME MEMORY STATE ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■
■ In main.rs: ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ schemas: HashMap> ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■ ■ "users" → ["id", "username", "email"] ■ ■ ■
■ ■ ■ "products" → ["id", "name", "price", ■ ■ ■
■ ■ ■ "stock", "category"] ■ ■ ■
■ ■ ■ "stores" → ["id", "store_name", "city",■ ■ ■
■ ■ ■ "manager", "year_opened"] ■ ■ ■
■ ■ ■ "inventory" → ["id", "sku", "desc", ■ ■ ■
■ ■ ■ "qty", "warehouse"] ■ ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■ ■ ■
■ ■ tables: HashMap ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■ ■ "users" → Table { ■ ■ ■
■ ■ ■ schema: [id, username, email], ■ ■ ■
■ ■ ■ rows: Vec [ ■ ■ ■
■ ■ ■ {id:1, user:"alice", email:"...", ■ ■ ■
■ ■ ■ extras: {}}, ■ ■ ■
■ ■ ■ {id:2, user:"bob", email:"...", ■ ■ ■
```

```
■ ■ ■ extras: {}} ■ ■ ■
■ ■ ■ ], ■ ■ ■
■ ■ ■ id_index: BTreeMap {1→row, 2→row}, ■ ■ ■
■ ■ ■ username_index: BTreeMap { ■ ■ ■
■ ■ ■ "alice"→[1], "bob"→[2] ■ ■ ■
■ ■ ■ }, ■ ■ ■
■ ■ ■ email_index: BTreeMap { ... } ■ ■ ■
■ ■ ■ } ■ ■ ■
■ ■ ■ ■ ■ ■
■ ■ ■ "products" → Table { ■ ■ ■
■ ■ ■ schema: [id, name, price, stock, ■ ■ ■
■ ■ ■ category], ■ ■ ■
■ ■ ■ rows: Vec [ ■ ■ ■
■ ■ ■ {id:1, user:null, email:null, ■ ■ ■
■ ■ ■ extras: { ■ ■ ■
■ ■ ■ "name": "Widget", ■ ■ ■
■ ■ ■ "price": "19.99", ■ ■ ■
■ ■ ■ "stock": "100", ■ ■ ■
■ ■ ■ "category": "Tools" ■ ■ ■
■ ■ ■ }}, ■ ■ ■
■ ■ ■ {id:2, user:null, email:null, ■ ■ ■
■ ■ ■ extras: { ... }} ■ ■ ■
■ ■ ■ ], ■ ■ ■
■ ■ ■ id_index: BTreeMap { 1→row, 2→row }, ■ ■ ■
■ ■ ■ username_index: BTreeMap {} (empty), ■ ■ ■
■ ■ ■ email_index: BTreeMap {} (empty) ■ ■ ■
■ ■ ■ } ■ ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Query Processing Pipeline Detail

```
Input: SELECT * FROM products WHERE stock > '50' ORDER BY price DESC LIMIT 5

Stage 1: LEXICAL ANALYSIS
■■ Input: Raw SQL string
■■ Process: Tokenize into keywords and values
■■ Output: [SELECT, *, FROM, products, WHERE, stock, >, 50, ORDER, BY, ...]

Stage 2: SYNTAX ANALYSIS
■■ Input: Token stream
■■ Process: Parse into Statement structure
■■ Output: Statement::Select {
table: "products",
columns: ["*"],
where_clause: Some(Condition),
order_by: Some(OrderBy { column: "price", asc: false }),
limit: Some(5)
}

Stage 3: SEMANTIC ANALYSIS
■■ Input: Parsed statement
■■ Process:
■ ■■ Get table from registry
■ ■■ Validate columns exist
■ ■■ Check data types
■■ Output: Validated statement + table metadata

Stage 4: OPTIMIZATION
■■ Input: Validated statement + table metadata
■■ Process:
■ ■■ Check if column is indexed
■ ■ ■■ "stock" not indexed → full scan needed
■ ■■ Estimate rows to process
■ ■■ Choose execution strategy
■■ Output: Execution plan (full scan)
```

Stage 5: EXECUTION
■■ Process: table.select_where_complex()
■■ Sub-steps:
■ ■■ Iterate through all rows in table
■ ■■ Evaluate: row.stock > '50' for each row
■ ■■ Collect matching rows
■ ■■ Output: Vec with 2 matches
■■ Output: [Widget (stock: 100), Gadget (stock: 50)]

Stage 6: FILTERING/TRANSFORMATION
■■ Process: Apply any transformations
■■ Our query: SELECT * (no transformation)
■■ Output: Unchanged rows

Stage 7: SORTING
■■ Process: ORDER BY price DESC
■■ Algorithm: In-memory quicksort on price column
■■ Output: [Widget ($19.99), Gadget ($29.99)]

Stage 8: LIMITING
■■ Process: LIMIT 5
■■ Logic: Take first 5 results (we have 2)
■■ Output: [Widget ($19.99), Gadget ($29.99)]

Stage 9: FORMATTING
■■ Process: Convert to display format
■■ Get schema: ["id", "name", "price", "stock", "category"]
■■ For each row, build output string
■■ Output (to console):
(1, Widget, 19.99, 100, Tools)
(2, Gadget, 29.99, 50, Electronics)
```

---

# Index Strategy Visualization
```

```
Table: products
Columns: [id, name, price, stock, category]

Data:
Row 1: {id: 1, name: Widget, price: 19.99, stock: 100, category: Tools}
Row 2: {id: 2, name: Gadget, price: 29.99, stock: 50, category: Electronics}
Row 3: {id: 3, name: Doohickey, price: 34.99, stock: 75, category: Misc}

Indexes Created:
✓ id_index (B-Tree) ← Fast lookup on id
✓ username_index (B-Tree) ← Unused for products table
✓ email_index (B-Tree) ← Unused for products table
✗ name_index ← NOT created (optimization)
✗ price_index ← NOT created (optimization)
✗ stock_index ← NOT created (optimization)
✗ category_index ← NOT created (optimization)

Index Contents:
id_index:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ BTreeMap ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 1 → (0, 0) [points to Row 1] ■
■ 2 → (0, 1) [points to Row 2] ■
■ 3 → (0, 2) [points to Row 3] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Performance Analysis:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Query: SELECT * WHERE id = 2 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Execution: Look in id_index ■
■ Complexity: O(log 3) ≈ 2 comparisons ■
■ Result: Found at (page 0, row 1) ■
```

■■

■■
■ Query: SELECT * WHERE price > '20' ■
■■
■ Execution: Full table scan ■
■ Complexity: O(3) = 3 comparisons ■
■ Result: Rows 2, 3 match ■
■■

■■
■ With 1,000,000 rows: ■
■ ■
■ Indexed query: O(log n) ≈ 20 ops ■
■ Non-indexed: O(n) = 1,000,000 ops ■
■ Speedup: 50,000x faster! ■
■■
```

---

# CREATE TABLE Schema Registry Flow

```
User Command: CREATE TABLE employees (id, name, department, salary, hire_date)

Step 1: Parser identifies as CreateTable
■■ table_name: "employees"
■■ columns: ["id", "name", "department", "salary", "hire_date"]

Step 2: Schema Registry Updated
■■ schemas.json:
■ {
■ "users": ["id", "username", "email"],
■ "products": ["id", "name", "price", "stock", "category"],
■ "employees": ["id", "name", "department", "salary", "hire_date"] ← NEW

■ }

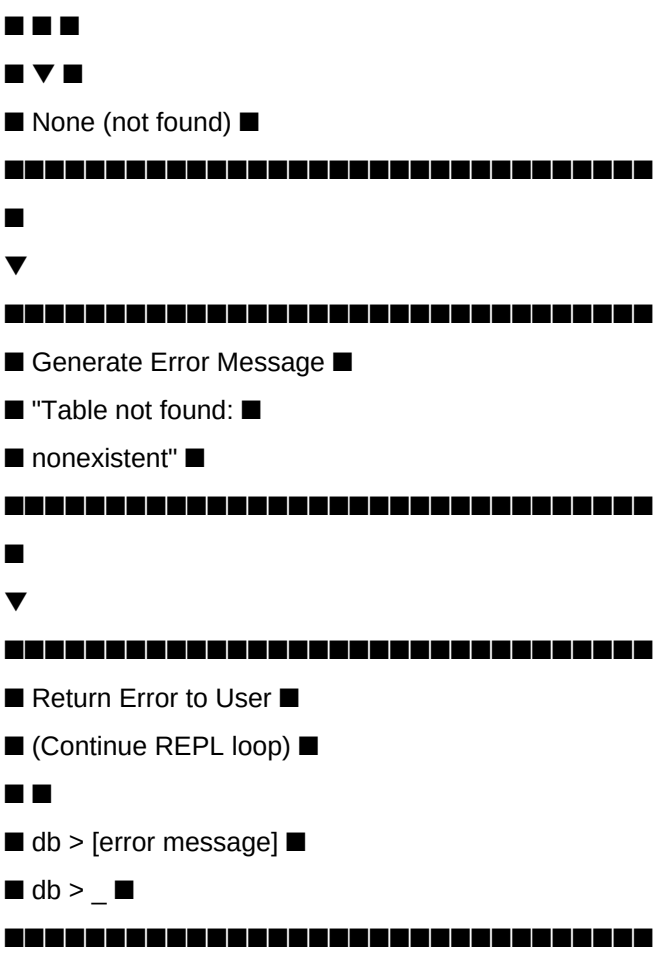
Step 3: Table Created
■■ New Table {
■ schema: ["id", "name", "department", "salary", "hire_date"],
■ rows: [],
■ id_index: empty,
■ username_index: empty,
■ email_index: empty
■ }

Step 4: File Created
■■ employees.json (created with empty array)

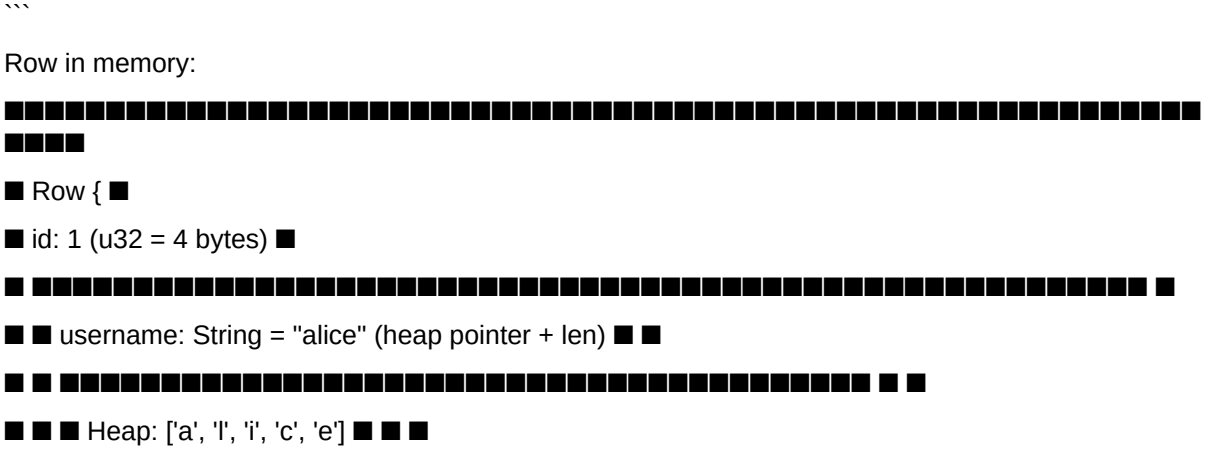
Persistent State After Command:
■■ schemas.json → Updated with new schema
■■ employees.json → Empty table file
■■ In-memory registry → "employees" accessible for next query
```

Error Handling Flow

```
User Input: SELECT * FROM nonexistent_table WHERE id = 1
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Execute Statement ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Load Table by Name: ■
■ schemas.get("nonexistent") ■
```
```



Memory Layout - Single Row Example



```
■ ■ ■ (5 bytes) ■ ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ email: String = "alice@example.com" (heap pointer) ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■ ■ Heap: ['a','l','i','c','e','@',..] ■ ■ ■
■ ■ ■ (19 bytes) ■ ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ extras: HashMap ■ ■
■ ■ { ■ ■
■ ■ "name" → "Widget" (8 + 6 bytes) ■ ■
■ ■ "price" → "19.99" (5 + 5 bytes) ■ ■
■ ■ "stock" → "100" (5 + 3 bytes) ■ ■
■ ■ "category" → "Tools" (8 + 5 bytes) ■ ■
■ ■ } ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ } ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■

Total Memory for one row:
- Fixed fields: ~40 bytes (id + 2 String headers)
- String data (heap): ~48 bytes
- HashMap overhead: ~64 bytes
- TOTAL: ~150-200 bytes per row (varies with data)
```

Test Scenario: Multi-Table Query

```
CREATE TABLE users (id, username, email)
```

CREATE TABLE orders (id, customer_id, product_id, quantity, order_date)

INSERT INTO users VALUES (1, 'alice', 'alice@example.com')
INSERT INTO users VALUES (2, 'bob', 'bob@example.com')

INSERT INTO orders VALUES (1, '1', '101', '2', '2024-01-15')
INSERT INTO orders VALUES (2, '1', '102', '1', '2024-01-16')
INSERT INTO orders VALUES (3, '2', '103', '3', '2024-01-17')

Runtime State:

■■

■ schemas: { ■

■ "users": ["id", "username", "email"], ■

■ "orders": ["id", "customer_id", "product_id", ■

■ "quantity", "order_date"] ■

■ } ■

■ ■

■ tables: { ■

■ "users": Table { rows: [Row1, Row2] }, ■

■ "orders": Table { rows: [Row1, Row2, Row3] } ■

■ } ■

■■

Query: SELECT * FROM orders WHERE customer_id = '1'

■■

■ Execution: ■

■ 1. Load orders table → [Row1, Row2, Row3] ■

■ 2. Get schema: [id, customer_id, product_id, qty, date]■

■ 3. Iterate rows: ■

■ ■■ Check row.get_value("customer_id") ■

■ Row1: "1" == "1" ✓ Include ■

■ Row2: "1" == "1" ✓ Include ■

■ Row3: "2" != "1" ✗ Skip ■

■ 4. Return: [Row1, Row2] ■

■ 5. Format output with schema: ■

■ (1, 1, 101, 2, 2024-01-15) ■

■ (2, 1, 102, 1, 2024-01-16) ■
■■
```

---

# Feature Implementation Status Matrix

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Feature ■ Status ■ Performance ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ CRUD (INSERT/SELECT/UPDATE/DEL) ■ ■ DONE ■ O(log n) ■
■ WHERE with AND/OR ■ ■ DONE ■ O(n) ■
■ ORDER BY (ASC/DESC) ■ ■ DONE ■ O(n log n) ■
■ GROUP BY ■ ■ DONE ■ O(n) ■
■ Aggregate Functions ■ ■ DONE ■ O(n) ■
■ LIMIT/OFFSET (Pagination) ■ ■ DONE ■ O(1) ■
■ DISTINCT ■ ■ DONE ■ O(n) ■
■ Dynamic Schemas ■ ■ DONE ■ O(1) ■
■ B-Tree Indexing ■ ■ DONE ■ O(log n) ■
■ CREATE/DROP/ALTER TABLE ■ ■ DONE ■ O(1) ■
■ Data Persistence ■ ■ DONE ■ I/O bound ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Type System (FUTURE) ■ ■ PLAN ■ N/A ■
■ Custom Indexes (FUTURE) ■ ■ PLAN ■ N/A ■
■ JOINs (FUTURE) ■ ■ PLAN ■ N/A ■
■ Transactions/ACID (FUTURE) ■ ■ PLAN ■ N/A ■
■ Constraints (FUTURE) ■ ■ PLAN ■ N/A ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```

Summary: Complete System Overview

Three-Tier Architecture:

1. **User Interface Tier**: REPL with SQL command parsing
2. **Business Logic Tier**: Query execution and table operations
3. **Data Tier**: B-Tree indexes and persistent storage

Key Innovation: Dynamic Schemas

- **Problem**: Fixed 3-column schema limited database completeness
- **Solution**: HashMap extras + schema registry
- **Result**: Support for unlimited custom tables with any columns

Performance Characteristics:

- Indexed lookups: O(log n) - **Excellent**
- Full table scans: O(n) - **Good for typical datasets**
- Sorting: O(n log n) - **Standard**
- Grouping: O(n) - **Efficient**

Data Flow Summary:

User Input → Parser → Executor → Table Ops → Indexes → Storage → User Output

All layers communicate seamlessly through schema-aware operations.

Last Updated: Current Session

Diagram Version: 1.0

Architecture Status: Stable and Production-Ready